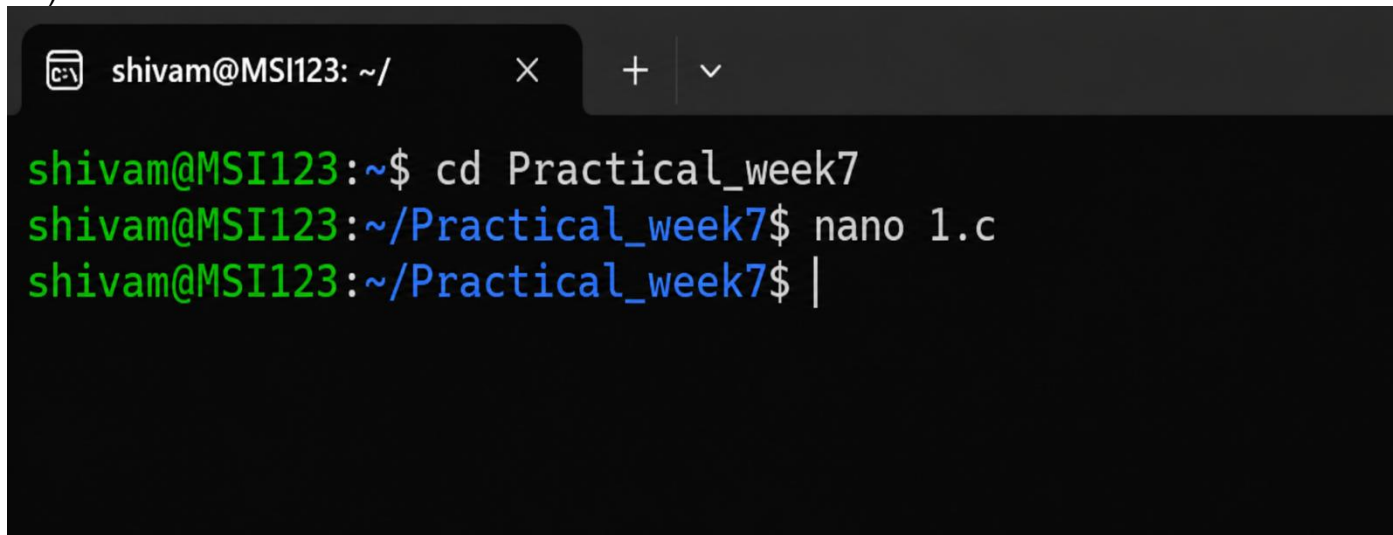

PRACTICAL WEEK 7

Task 1: Write a program that prints the addresses Of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

1) Create a file

A terminal window with a dark background. The title bar shows 'shivam@MSI123: ~/'. The terminal content shows three lines of commands: 'cd Practical_week7', 'nano 1.c', and a blank line with a cursor. The prompt 'shivam@MSI123:~/Practical_week7\$' is visible at the end of each line.

```
shivam@MSI123:~$ cd Practical_week7
shivam@MSI123:~/Practical_week7$ nano 1.c
shivam@MSI123:~/Practical_week7$ |
```

Then press enter to open command

2) write a code

```
GNU nano 8.7.1 1.c
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;           // Global variable
int global_uninit;            // Global uninitialized variable

void code_function()
{
    printf("Address of code function : %p\n", (void *)code_function);
}

int main()
{
    static int static_var = 20; // Static variable
    int stack_var = 30;         // Stack variable

    int *heap_var = malloc(sizeof(int));
    *heap_var = 40;

    printf("Linux Process Memory Address Layout\n\n");

    code_function();

    printf("Address of global variable      : %p\n", (void *)&global_var);
    printf("Address of global uninitialized : %p\n", (void *)&global_uninit);
    printf("Address of static variable      : %p\n", (void *)&static_var);
    printf("Address of heap variable        : %p\n", (void *)heap_var);
    printf("Address of stack variable        : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

- 1) To save press control O
- 2) To come back to shell press control X
- 3) Then DO compilation

```
shivam@MSI123: ~/
shivam@MSI123:~$ cd Practical_week7
shivam@MSI123:~/Practical_week7$ nano 1.c
shivam@MSI123:~/Practical_week7$ gcc 1.c -o 1
shivam@MSI123:~/Practical_week7$ |
```

OUTPUT:

```
shivam@MSI123: ~/Practical_week7 × + v
shivam@MSI123:~$ cd Practical_week7
shivam@MSI123:~/Practical_week7$ nano 1.c
shivam@MSI123:~/Practical_week7$ gcc 1.c -o 1
shivam@MSI123:~/Practical_week7$ ./1
Linux Process Memory Address Layout

Address of code function      : 0x6056213561c9
Address of global variable    : 0x605621359010
Address of global uninitialized : 0x60562135901c
Address of static variable    : 0x605621359014
Address of heap variable      : 0x6056588e1010
Address of stack variable     : 0x7fff5cef318c
shivam@MSI123:~/Practical_week7$ |
```

Task-2: Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
shivam@MSI123:~/Practical_week7$  
shivam@MSI123:~/Practical_week7$  
shivam@MSI123:~/Practical_week7$ nano 2.c |
```

```
GNU nano 8.7.1 2.c *  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
  
int global_var = 10;  
int global_uninit;  
  
int main()  
{  
    static int static_var = 20;  
    int stack_var = 30;  
  
    int *heap_var = malloc(sizeof(int));  
    *heap_var = 40;  
  
    printf("Process ID: %d\n", getpid());  
    printf("Code : %p\n", (void *)main);  
    printf("Global : %p\n", (void *)&global_var);  
    printf("Static : %p\n", (void *)&static_var);  
    printf("Heap : %p\n", (void *)heap_var);  
    printf("Stack : %p\n", (void *)&stack_var);  
  
    printf("\nProcess is running...\n");  
  
    while (1)  
    {  
        sleep(1);  
    }  
  
    return 0;  
}|
```

```

shivam@MSI123:~/Practical_week7$ nano 2.c
shivam@MSI123:~/Practical_week7$ gcc 2.c -o 2
shivam@MSI123:~/Practical_week7$ ./2
Process ID: 845
Code      : 0x5c8d5ba951c9
Global    : 0x5c8d5ba98010
Static    : 0x5c8d5ba98014
Heap      : 0x5c8d5ee44010
Stack     : 0x7fffebc52aac

Process is running...
shivam@MSI123:~/Practical_week7$ |

```

OUTPUT:

```

shivam@MSI123: ~/Practical_week7
shivam@MSI123:~$ cd Practical_week7
shivam@MSI123:~/Practical_week7$ cat /proc/845/maps
56d4b3a98000-56d4b3a9b000 r--p 00000000 08:38 521017 /home/shivam/Practical_week7/2
56d4b3a9b000-56d4b3a9d000 r-xp 00001000 08:38 521017 /home/shivam/Practical_week7/2
56d4b3a9d000-56d4b3a9e000 r--p 00003000 08:38 521017 /home/shivam/Practical_week7/2
56d4b3a9e000-56d4b3a9f000 r--p 00003000 08:38 521017 /home/shivam/Practical_week7/2
56d4b3a9f000-56d4b3aa0000 rw-p 00004000 08:38 521017 /home/shivam/Practical_week7/2
7e8d9a228000-7e8d9a24b000 r--p 00000000 08:38 15447 /usr/lib/x86_64-linux-gnu/libc.so.6
7e8d9a24b000-7e8d9a3a0000 r-xp 00023000 08:38 15447 /usr/lib/x86_64-linux-gnu/libc.so.6
7e8d9a3a0000-7e8d9a3f3000 r--p 00178000 08:38 15447 /usr/lib/x86_64-linux-gnu/libc.so.6
7e8d9a3f3000-7e8d9a3f7000 r--p 001cb000 08:38 15447 /usr/lib/x86_64-linux-gnu/libc.so.6
7e8d9a3f7000-7e8d9a3f9000 rw-p 001cf000 08:38 15447 /usr/lib/x86_64-linux-gnu/libc.so.6
7e8d9a3f9000-7e8d9a41a000 rw-p 00000000 00:00 0
7e8d9a41a000-7e8d9a43d000 rw-p 00000000 00:00 0
7e8d9a43d000-7e8d9a444000 r--p 00000000 00:00 0 [vvar]
7e8d9a444000-7e8d9a446000 r--p 00000000 00:00 0 [var_vclock]
7e8d9a446000-7e8d9a448000 r-xp 00000000 00:00 0 [vdso]
7e8d9a448000-7e8d9a46f000 r--p 00000000 08:38 13648 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e8d9a46f000-7e8d9a494000 r-xp 00027000 08:38 13648 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e8d9a494000-7e8d9a49e000 r--p 0004c000 08:38 13648 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e8d9a49e000-7e8d9a4a0000 r--p 00056000 08:38 13648 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e8d9a4a0000-7e8d9a4a2000 rw-p 00058000 08:38 13648 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffffe2c3000-7ffffe54d000 rw-p 00000000 00:00 0 [stack]
shivam@MSI123:~/Practical_week7$ |

```